

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

“Программирование”

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Вариант 9-1-1-5

Выполнила:

Студентка гр. 3148 Жук Анастасия



Проверил:

Волков А. Г.

Санкт-Петербург
2024г.

Задание

Разработать на языке C для ОС Linux программу, которая читает данные из стандартного потока ввода или файла, осуществляет преобразование в соответствии с вариантом лабораторной работы и выводит результат в стандартный поток вывода или файл.

Последовательности: ISBN-13 (с проверкой по алгоритму ISBN-13).

Примеры: 978-0-306-40615-7, 978-2-266-11156-0

Дополнительные опции:

-b=M

Опция -b задает номер начальной строки (строки нумеруются с единицы). Преобразование выполняется, начиная со строки M. Если опция -b не указана, то преобразование начинается с первой строки.

-e=N

Опция -e задает номер конечной строки. Преобразование выполняется до строки N включительно. Если опция -e не указана, преобразование продолжается до последней строки.

Разметка Markdown, используемая для выделения найденных фрагментов текста: курсив (*Пример*)

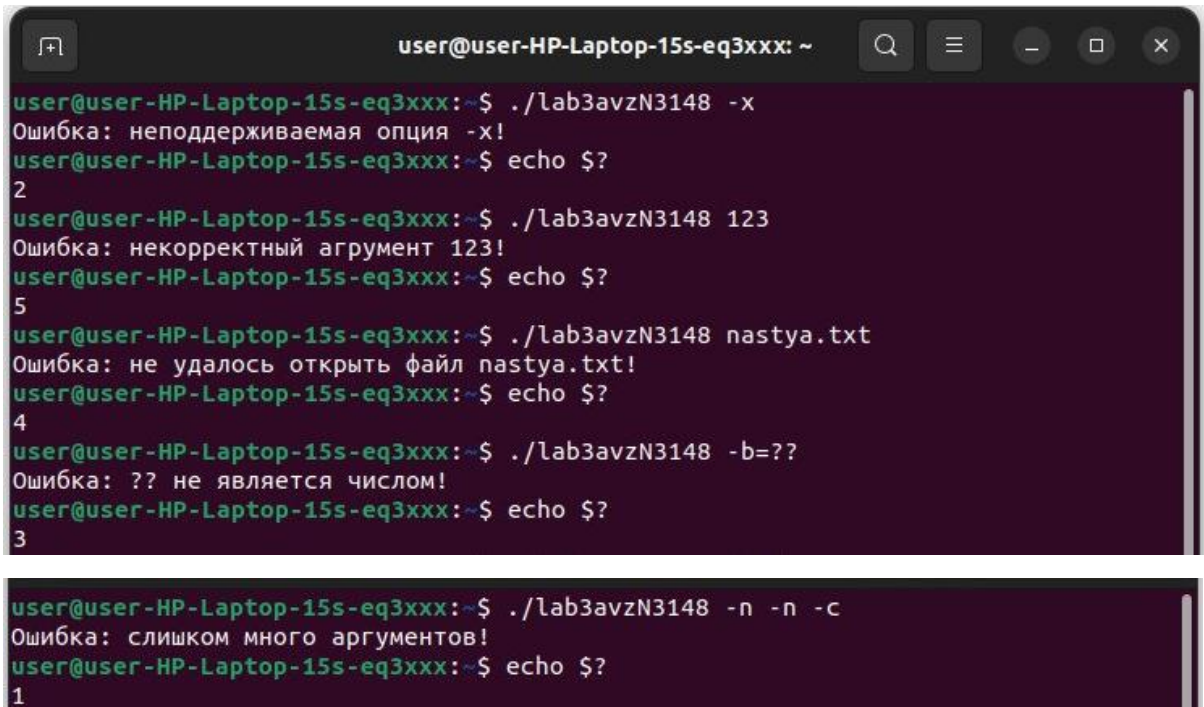
Цвет для выделения найденных фрагментов текста при выводе с опцией -c: пурпурный (magenta) (35)

Make-файл

```
1 .PHONY: all clean
2
3 APP=lab3avzN3148
4 CFLAGS=-Wall -Wextra -Werror -g
5 CCON1=infileall
6 CCON2=infileonly
7 CF1=inconsolall
8 CF2=inconsolonly
9
10 all: $(APP)
11
12 $(APP):
13     gcc -c $(CCON1).c $(CCON2).c $(CF1).c $(CF2).c
14     gcc -o $(APP) $(CFLAGS) $(APP).c $(CCON1).o $(CCON2).o $(CF1).o $(CF2).o
15
16 clean:
17     rm $(APP) $(CCON1).o $(CCON2).o $(CF1).o $(CF2).o
18 |
```

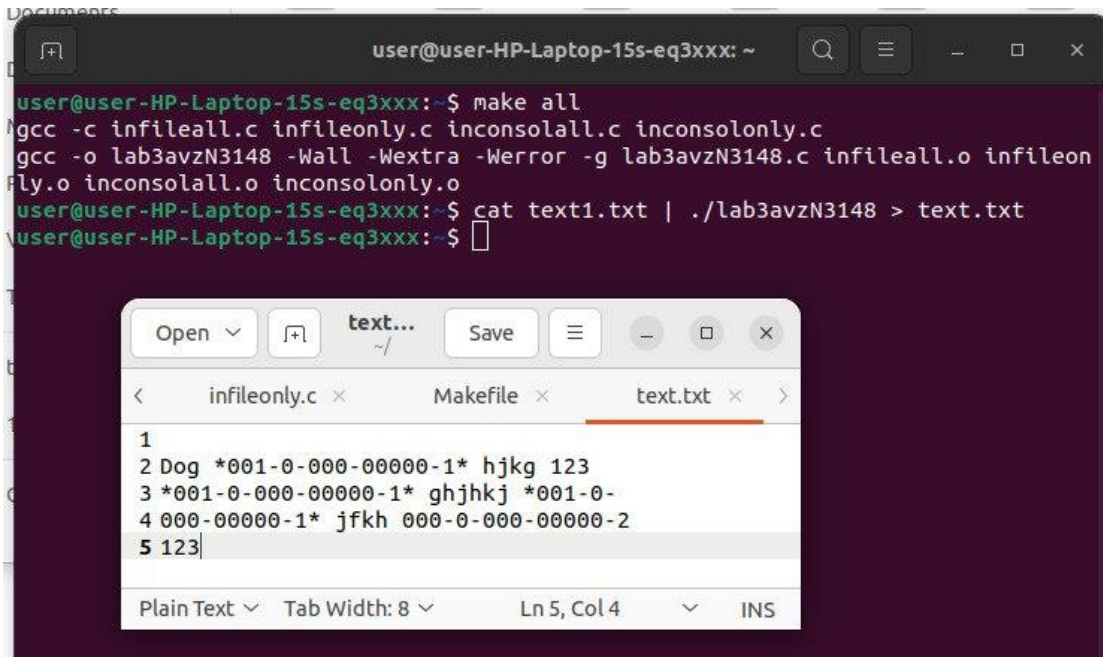
Примеры работы программы

Обработка ошибок:



```
user@user-HP-Laptop-15s-eq3xxx: ~  
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -x  
Ошибка: неподдерживаемая опция -x!  
user@user-HP-Laptop-15s-eq3xxx:~$ echo $?  
2  
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 123  
Ошибка: некорректный аргумент 123!  
user@user-HP-Laptop-15s-eq3xxx:~$ echo $?  
5  
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 nastya.txt  
Ошибка: не удалось открыть файл nastya.txt!  
user@user-HP-Laptop-15s-eq3xxx:~$ echo $?  
4  
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -b=??  
Ошибка: ?? не является числом!  
user@user-HP-Laptop-15s-eq3xxx:~$ echo $?  
3  
  
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -n -n -c  
Ошибка: слишком много аргументов!  
user@user-HP-Laptop-15s-eq3xxx:~$ echo $?  
1
```

Примеры



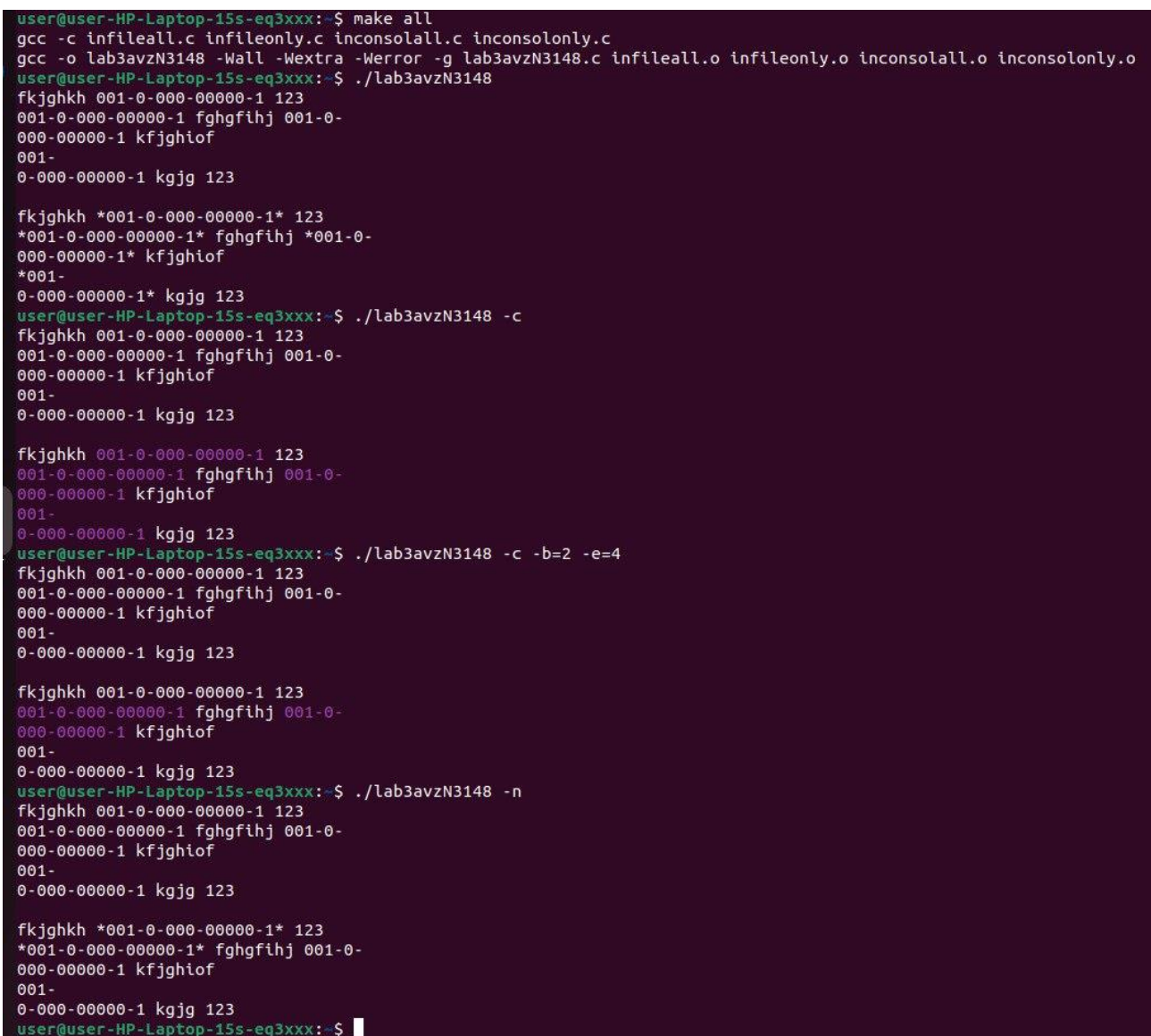
The screenshot shows a terminal window with the following commands and output:

```
user@user-HP-Laptop-15s-eq3xxx:~$ make all
gcc -c infileall.c infileonly.c inconsolall.c inconsolonly.c
gcc -o lab3avzN3148 -Wall -Wextra -Werror -g lab3avzN3148.c infileall.o infileonly.o inconsolall.o inconsolonly.o
user@user-HP-Laptop-15s-eq3xxx:~$ cat text1.txt | ./lab3avzN3148 > text.txt
user@user-HP-Laptop-15s-eq3xxx:~$
```

Below the terminal, a text editor window titled 'text.txt' is open, showing the following content:

```
1
2 Dog *001-0-000-00000-1* hjkg 123
3 *001-0-000-00000-1* ghjhkj *001-0-
4 000-00000-1* jfkh 000-0-000-00000-2
5 123|
```

The text editor also shows a status bar at the bottom: 'Plain Text', 'Tab Width: 8', 'Ln 5, Col 4', and 'INS'.



The screenshot shows a terminal window with the following commands and output:

```
user@user-HP-Laptop-15s-eq3xxx:~$ make all
gcc -c infileall.c infileonly.c inconsolall.c inconsolonly.c
gcc -o lab3avzN3148 -Wall -Wextra -Werror -g lab3avzN3148.c infileall.o infileonly.o inconsolall.o inconsolonly.o
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh *001-0-000-00000-1* 123
*001-0-000-00000-1* fghgfi hj *001-0-
000-00000-1* kfjghiof
*001-
0-000-00000-1* kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -c
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -c -b=2 -e=4
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -n
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh *001-0-000-00000-1* 123
*001-0-000-00000-1* fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$
```

```

user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -n -c
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 -c -b=2 -e=4 -n
fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123

fkjghkh 001-0-000-00000-1 123
001-0-000-00000-1 fghgfi hj 001-0-
000-00000-1 kfjghiof
001-
0-000-00000-1 kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt
fkjghkh *001-0-000-00000-1* 123
*001-0-000-00000-1* fghgfi hj *001-0-
000-00000-1* kfjghiof
*001-
0-000-00000-1* kgjg 123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt
Dog *001-0-000-00000-1* hjkg 123
*001-0-000-00000-1* ghjhkj *001-0-
000-00000-1* jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$ █

```

text1.txt

```

1 Dog 001-0-000-00000-1 hjkg 123
2 001-0-000-00000-1 ghjhkj 001-0-
3 000-00000-1 jfkh 000-0-000-00000-2
4 123|

```



```

user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt -n
Dog *001-0-000-00000-1* hjkg 123
*001-0-000-00000-1* ghjhkj 001-0-
000-00000-1 jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt text.txt
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt text.txt -n
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt text.txt -n
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt -c
Dog 001-0-000-00000-1 hjkg 123
001-0-000-00000-1 ghjhkj 001-0-
000-00000-1 jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt -c -n
Dog 001-0-000-00000-1 hjkg 123
001-0-000-00000-1 ghjhkj 001-0-
000-00000-1 jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt -c -e=2
Dog 001-0-000-00000-1 hjkg 123
001-0-000-00000-1 ghjhkj 001-0-
000-00000-1 jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$ ./lab3avzN3148 text1.txt -c -n -b=2
Dog 001-0-000-00000-1 hjkg 123
001-0-000-00000-1 ghjhkj 001-0-
000-00000-1 jfkh 000-0-000-00000-2
123
user@user-HP-Laptop-15s-eq3xxx:~$

```

text.txt (./lab3avzN3148 text1.txt text.txt)

```

1 Dog *001-0-000-00000-1* hjkg 123
2 *001-0-000-00000-1* ghjhkj *001-0-
3 000-00000-1* jfkh 000-0-000-00000-2
4 123

```

text.txt (./lab3avzN3148 text1.txt text.txt -n)

text.txt	×	inconsolall.c
1 Dog *001-0-000-00000-1* hjkg 123		
2 *001-0-000-00000-1* ghjhkj 001-0-		
3 000-00000-1* jfkh 000-0-000-00000-2		
4 123		

text.txt (./lab3avzN3148 text1.txt text.txt -c)

text.txt	×	inconsolall.c	×
1 Dog [35m001-0-000-00000-1[m hjkg 123			
2 [35m001-0-000-00000-1[m ghjhkj [35m001-0-			
3 000-00000-1[m jfkh 000-0-000-00000-2			
4 123			

text.txt (./lab3avzN3148 text1.txt text.txt -c -n)

text.txt	×	inconsolall.c	×
1 Dog [35m001-0-000-00000-1[m hjkg 123			
2 [35m001-0-000-00000-1[m ghjhkj 001-0-			
3 000-00000-1 jfkh 000-0-000-00000-2			
4 123			

Программа

Вся программа состоит из 5 подпрограмм:

- lab3avzN3148.c – содержит обработку ошибок и определяет, в какой из файлов, в которых находятся функции, нужно запустить.
- inconsolall.c – обрабатывает текст, введенный с клавиатуры и запускается с помощью утилиты cat, выделяет все встречающиеся последовательности.
- inconsolonly.c – обрабатывает текст, введенный с клавиатуры, выделяет только последовательности, расположенные целиком в одной строке.
- infileall.c – обрабатывает текст файла, выделяет все встречающиеся последовательности.
- infileonly.c – обрабатывает текст файла, выделяет только последовательности, расположенные целиком в одной строке.

lab3avzN3148.c:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>

extern int errno;

//виды ошибок
#define NO_ERROR 0
#define ERROR_MANY_ARGUMENTS 1
#define ERROR_INCORRECT_OPTIONS 2
#define ERROR_NOT_DIGITALS 3
#define ERROR_DONT_OPEN_FILE 4
#define ERROR_WRONG_ARGUMENTS 5

void infileall(char *, char *, int, int, int);
void infileonly(char *, char *, int, int, int);
void inconsolall(int, int, int);
void inconsolonly(int, int, int);

int main(int argc, char *argv[]) {

    char *DEBUG = getenv("LAB3DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    int c = 0, b = 1, e = 0, n = 0; //c = 1, b = k, e = m, n = 1
    char *txt1 = NULL;
    txt1 = (char*) malloc(sizeof(char));
```

```

char *txt2 = NULL;
txt2 = (char*) malloc(sizeof(char)); //txt1 = 1, txt2 = 1, no = 1
int c1 = 0, c2 = 0, c3 = 0;
errno = NO_ERROR;
//проверка
if (argc == 2 && strcmp(argv[1], "-v") == 0) {
    printf("Жук Анастасия Валерьевна гр. N3148\nВариант: 9-1-1-5\n");
    return errno;
}
else if (argc >= 2) {
    for (int i = 1; i < argc; i++) {
        if (strchr(argv[i], '.') != NULL) {
            if (strlen(txt1) == 0) {
                txt1 = (char*) realloc(txt1, strlen(argv[i]) * sizeof(char));
                memcpy(txt1, argv[i], strlen(argv[i]));
            }
            else {
                txt2 = (char*) realloc(txt2, strlen(argv[i]) * sizeof(char));
                memcpy(txt2, argv[i], strlen(argv[i]));
                c3++;
            }
        }
        else if (strcmp(argv[i], "-c") == 0) c++;
        else if (strcmp(argv[i], "-n") == 0) n++;
        else if (argv[i][0] == '-' && argv[i][1] == 'b') {
            if (sscanf(strchr(argv[i], '=') + 1, "%d", &b) == 0) {
                printf("Ошибка: ");
                for (int j = 3; j < (int) strlen(argv[i]); j++) {
                    printf("%c", argv[i][j]);
                }
                printf(" не является числом!\n");
                return ERROR_NOT_DIGITALS;
            }
        }
        c1++;
    }
    else if (argv[i][0] == '-' && argv[i][1] == 'e') {
        if (sscanf(strchr(argv[i], '=') + 1, "%d", &e) == 0) {
            printf("Ошибка: ");
            for (int j = 3; j < (int) strlen(argv[i]); j++) {
                printf("%c", argv[i][j]);
            }
            printf(" не является числом!\n");
            errno = ERROR_NOT_DIGITALS;
        }
        c2++;
    }
    else if (argv[i][0] == '-') {
        printf("Ошибка: неподдерживаемая опция ");
    }
}

```



```

        for (int j = 0; j < (int) strlen(argv[i]); j++) {
            if (argv[i][j] == '=') break;
            printf("%c", argv[i][j]);
        }
        printf("\n");
        errno = ERROR_INCORRECT_OPTIONS;
    }
    else {
        printf("Ошибка: некорректный аргумент %s!\n", argv[i]);
        errno = ERROR_WRONG_ARGUMENTS;
    }
}
if (c3 >= 2 || c >= 2 || n >= 2 || c1 >= 2 || c2 >= 2) {
    printf("Ошибка: слишком много аргументов!\n");
    errno = ERROR_MANY_ARGUMENTS;
}
}

if (errno == NO_ERROR) {
    if (strlen(txt1) != 0) {
        FILE *f = fopen(txt1, "r");
        if (!f) {
            printf("Ошибка: не удалось открыть файл %s!\n", txt1);
            errno = ERROR_DONT_OPEN_FILE;
            return errno;
        }
        else fclose(f);
        if (n == 0) infileall(txt1, txt2, c, b, e);
        else infileonly(txt1, txt2, c, b, e);
    }
    else {
        if (n != 0) inconsolonly(c, b, e);
        else inconsolall(c, b, e);
    }
}
return errno;
}

```

inconsolall.c:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

```

//обрабатывает все подходящие

```
void inconsolall(int cv, int bv, int ev) {
```

//str - вводимая строка, str1 - вспомогательная строка, которая будет хранить в себе
нужную нам подстроку (или что-то похожее на неё)

```
    char *str = NULL;
```

```

char *str1 = NULL;
str = (char*) malloc(sizeof(char));
str1 = (char*) malloc(sizeof(char));
//для считывания символов
int t;
char c;
int i = 0, k = 0, r = 0; //i - текущая длина str, k - текущая длина str1
int y = 0, g = 0; //счетчики
char *arr = NULL; //нужна для того, чтобы хранить переносы строк между частями
одной последовательности
arr = (char*) malloc(sizeof(char));
int w = 0; //вспомогательная переменная, чтобы записывать символы в строку str
int flag = 1; //флаг для обработки последовательностей
int countn = 1; //счетчик переносов строк
while ((t = getchar()) != EOF) {
    //обработка переноса строки и других символов
    if (t == 13) continue;
    else if (t == 10) c = '\n';
    else c = (char) t;
    i++;
    str = (char*) realloc(str, i * sizeof(char)); //увеличим строку
    if ((bv == 1 || countn >= bv) && (ev == 0 || ev >= countn)) { //обработка
конкретных строк (ограничения b и e)
        if ((isdigit(c) || c == '-' || (c == '\n' && k > 0 && k < 17))) { //будем здесь
собирать строки с числами
            if (c == '\n') {
                countn++;
                r++;
                arr = (char*) realloc(arr, r * sizeof(char)); //увеличим
строку
                arr[r - 1] = c;
            }
            else {
                k++;
                r++;
                str1 = (char*) realloc(str1, k * sizeof(char)); //увеличим
строку
                str1[k - 1] = c;
                arr = (char*) realloc(arr, r * sizeof(char)); //увеличим
строку
                arr[r - 1] = c;
                if (c == '-') y++; //счетчик символов '-'
                //обработка для того, чтобы правильно считывать
последовательности типа "xxx..x\nxxx-x-xxx-xxxxx-x"
                if (k == 4 && c != '-') {
                    for (int j = 0; j < k - 1; j++) str1[j] = str1[j +
1];
                    str1[k - 1] = '-';

```

```

        k--;
        str1 = (char*) realloc(str1, k * sizeof(char));
        flag = 0;
    }
    else if (k == 4 && c == '-' && r > 4 && arr[r - 5] == '\n') {
        for (int j = 0; j < r - 4; j++) {
            str[w] = arr[j];
            w++;
        }
        explicit_bzero(arr, 0);
        r = 4;
        arr = (char*) realloc(arr, r * sizeof(char));
        memcpy(arr, str1, 4);
        flag = 1;
    }
}
}
else {
    if (c == '\n') countn++;
    if (y == 4 && k == 17 && (c == ' ' || c == '\n') && flag) {
//проверка, что строка str1 вида xxx-x-xxx-xxxxx-x
        char *str2 = NULL; //вспомогательная строка (её будем
разбивать по "-")

        str2 = (char*) realloc(str2, k * sizeof(char));
        memcpy(str2, str1, k);
        char *p = strtok(str2, "-");
        int f = 1;
        while (p) {
            if (g == 0 && strlen(p) != 3) f = 0;
            else if (g == 1 && strlen(p) != 1) f = 0;
            else if (g == 2 && strlen(p) != 3) f = 0;
            else if (g == 3 && strlen(p) != 5) f = 0;
            g++;
            if (f == 0) break;
            else p = strtok(NULL, "-");
        }
        explicit_bzero(str2, 0);
        free(str2);
        if (f == 1) { //проверка по алгоритму ISBN-13
            int d = 0, l = 0;
            for (int j = 0; j < 15; j++) {
                if (j != 3 && j != 5 && j != 9) {
                    l++; //для того, чтобы умножать на
3 только четные элементы

                    d += 2 * ((l + 1) % 2) * ((int) str1[j] -
48) + (int) str1[j] - 48; //48 - код 0 в ASCII
                }
            }
        }
    }
}

```

```

    }
    if (d % 10 != ((int) str1[16] - 48)) f = 0;
    else { //нужно увеличить строку str на 2 для того,
чтобы можно было обособить подходящую нам str1, и вставить ее в исходную
        i += 2;
        str = (char*) realloc(str, i * sizeof(char));
        str[i - 2 - r - 1] = '*';
        memcpy(str + i - 2 - r, arr, r);
        str[i - 2] = '*';
        str[i - 1] = c;
        w += 3 + r;
        //очистим str1
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
        //очистим arr
        explicit_bzero(arr, 0);
        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
}
if (f == 0) {
    //записываем в str то, что лежит в str1 (так как так
лежит то, что нам не подошло)
    memcpy(str + i - r - 1, arr, r);
    str[i - 1] = c;
    w += r + 1;
    //очистим str1
    explicit_bzero(str1, 0);
    str1 = (char*) realloc(str1, sizeof(char));
    k = 0;
    //очистим arr
    explicit_bzero(arr, 0);
    arr = (char*) realloc(arr, sizeof(char));
    r = 0;
}
}
else { //записываем в str то, что лежит в str1 (так как так лежит
то, что нам не подошло)
    memcpy(str + i - r - 1, arr, r);
    str[i - 1] = c;
    w += r + 1;
    //очистим str1
    explicit_bzero(str1, 0);
    str1 = (char*) realloc(str1, sizeof(char));
    k = 0;
    //очистим arr
    explicit_bzero(arr, 0);

```

```

        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
    y = 0;
}
}
else {
    if (r != 0) {
        memcpy(str + i - r - 1, arr, r);
        w += r;
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
        explicit_bzero(arr, 0);
        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
    str[i - 1] = c;
    w++;
    if (c == '\n') countn++;
}
}
if (r != 0) memcpy(str + w, arr, r);
//покраска
if (cv != 0) {
    char *pp = strtok(str, "*");
    int a;
    if (str[0] == '*') a = 0;
    else a = 1;
    printf("\n");
    while (pp) {
        a++;
        if (a % 2 != 0) printf("\e[35m%s\e[m", pp);
        else printf("%s", pp);
        pp = strtok(NULL, "*");
    }
}
else printf("\n%s", str);
explicit_bzero(str, 0);
explicit_bzero(str1, 0);
explicit_bzero(arr, 0);
free(str);
free(str1);
free(arr);
}

```

inconsolonly.c:

```

#include <stdio.h>
#include <string.h>

```



```
#include <stdlib.h>
```

```
#include <ctype.h>
```

```
//обрабатывает только целые последовательности
```

```
void inconsonly(int cv, int bv, int ev) {
```

```
    //str - вводимая строка, str1 - вспомогательная строка, которая будет хранить в себе  
    //нужную нам подстроку (или что-то похожее на неё)
```

```
    char *str = NULL;
```

```
    char *str1 = NULL;
```

```
    str = (char*) malloc(sizeof(char));
```

```
    str1 = (char*) malloc(sizeof(char));
```

```
    //для считывания символов
```

```
    int t;
```

```
    char c;
```

```
    int i = 0, k = 0; //i - текущая длина str, k - текущая длина str1
```

```
    int y = 0, g = 0; //счетчики
```

```
    int countn = 1; //счетчик переносов строки, чтобы следить за номером строки
```

```
    while ((t = getchar()) != EOF) {
```

```
        //обработка переноса строки и других символов
```

```
        if (t == 13) continue;
```

```
        else if (t == 10) c = '\n';
```

```
        else c = (char) t;
```

```
        i++;
```

```
        str = (char*) realloc(str, i * sizeof(char)); //увеличим строку
```

```
        if ((bv == 1 || countn >= bv) && (ev == 0 || ev >= countn)) { //обработка  
        конкретных строк (ограничения b и e)
```

```
            if (isdigit(c) || c == '-') { //будем здесь собирать строки с числами
```

```
                k++;
```

```
                str1 = (char*) realloc(str1, k * sizeof(char)); //увеличим строку
```

```
                str1[k - 1] = c;
```

```
                if (c == '-') y++;
```

```
            }
```

```
            else {
```

```
                if (c == '\n') countn++;
```

```
                if (y == 4 && k == 17 && (c == ' ' || c == '\n')) { //проверка, что  
                строка str1 вида xxx-x-xxx-xxxxx-x
```

```
                char *str2 = NULL; //вспомогательная строка (её будем  
                разбивать по "-")
```

```
                str2 = (char*) realloc(str2, k * sizeof(char));
```

```
                memcpy(str2, str1, k);
```

```
                char *p = strtok(str2, "-");
```

```
                int f = 1;
```

```
                while (p) {
```

```
                    if (g == 0 && strlen(p) != 3) f = 0;
```

```
                    else if (g == 1 && strlen(p) != 1) f = 0;
```

```
                    else if (g == 2 && strlen(p) != 3) f = 0;
```

```
                    else if (g == 3 && strlen(p) != 5) f = 0;
```

```
                    g++;
```

```

        if (f == 0) break;
        else p = strtok(NULL, "-");

    }
    free(str2);
    if (f == 1) { //проверка по алгоритму ISBN-13
        int d = 0, l = 0;
        for (int j = 0; j < 15; j++) {
            if (j != 3 && j != 5 && j != 9) {
                l++; //для того, чтобы умножать на
3 только четные элементы
                d += 2 * ((l + 1) % 2) * ((int) str1[j] -
48) + (int) str1[j] - 48; //48 - код 0 в ASCII
            }
        }
        if (d % 10 != ((int) str1[16] - 48)) f = 0;
        else { //нужно увеличить строку str на 2 для того,
чтобы можно было обособить подходящую нам str1, и вставить ее в исходную
            i += 2;
            str = (char*) realloc(str, i * sizeof(char));
            str[i - 2 - k - 1] = '*';
            memcpy(str + i - 2 - k, str1, k);
            str[i - 2] = '*';
            str[i - 1] = c;
            //очистим str1
            explicit_bzero(str1, 0);
            str1 = (char*) realloc(str1, sizeof(char));
            k = 0;
        }
    }
    if (f == 0) {
        //записываем в str то, что лежит в str1 (так как так
лежит то, что нам не подошло)
        memcpy(str + i - k - 1, str1, k);
        str[i - 1] = c;
        //очистим str1
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
    }
}
else { //записываем в str то, что лежит в str1 (так как так лежит
то, что нам не подошло)
    memcpy(str + i - k - 1, str1, k);
    str[i - 1] = c;
    //очистим str1
    explicit_bzero(str1, 0);
    str1 = (char*) realloc(str1, sizeof(char));
}

```

```

                                k = 0;
                                }
                                y = 0;
                            }
                        }
                    else {
                        str[i - 1] = c;
                        if (c == '\n') countn++;
                    }
                }
            //покраска
            if (cv != 0) {
                char *pp = strtok(str, "*");
                int a;
                if (str[0] == '*') a = 0;
                else a = 1;
                printf("\n");
                while (pp) {
                    a++;
                    if (a % 2 != 0) printf("\e[35m%s\e[m", pp);
                    else printf("%s", pp);
                    pp = strtok(NULL, "*");
                }
            }
            else printf("\n%s", str);
            explicit_bzero(str, 0);
            explicit_bzero(str1, 0);
            free(str);
            free(str1);
        }
    }
}

```

infileall.c:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

//обрабатывает все подходящие последовательности (из файла)
void infileall(char *txt1, char *txt2, int cv, int bv, int ev) {
    FILE *f = fopen(txt1, "r");
    //str - вводимая строка, str1 - вспомогательная строка, которая будет хранить в себе
    //нужную нам подстроку (или что-то похожее на неё)
    char *str = NULL;
    char *str1 = NULL;
    str = (char*) malloc(sizeof(char));
    str1 = (char*) malloc(sizeof(char));
    //для считывания символов
    int t;
    char c;
}

```

```

int i = 0, k = 0, r = 0; //i - текущая длина str, k - текущая длина str1
int y = 0, g = 0; //счетчики
char *arr = NULL; //нужна для того, чтобы хранить переносы строк между частями
одной последовательности
arr = (char*) malloc(sizeof(char));
int w = 0; //вспомогательная переменная, чтобы записывать символы в строку str
int flag = 1; //флаг для обработки последовательностей
int countn = 1; //счетчик переносов строк
while ((t = fgetc(f)) != EOF) {
    //обработка переноса строки и других символов
    if (t == 13) continue;
    else if (t == 10) c = '\n';
    else c = (char) t;
    i++;
    str = (char*) realloc(str, i * sizeof(char)); //увеличим строку
    if ((bv == 1 || countn >= bv) && (ev == 0 || ev >= countn)) { //обработка
конкретных строк (ограничения b и e)
        if ((isdigit(c) || c == '-' || (c == '\n' && k > 0 && k < 17))) { //будем здесь
собирать строки с числами
            if (c == '\n') {
                countn++;
                r++;
                arr = (char*) realloc(arr, r * sizeof(char)); //увеличим
строку
                arr[r - 1] = c;
            }
            else {
                k++;
                r++;
                str1 = (char*) realloc(str1, k * sizeof(char)); //увеличим
строку
                str1[k - 1] = c;
                arr = (char*) realloc(arr, r * sizeof(char)); //увеличим
строку
                arr[r - 1] = c;
                if (c == '-') y++; //счетчик символов '-'
                //обработка для того, чтобы правильно считывать
последовательности типа "xxx..x\nxxx-x-xxx-xxxxx-x"
                if (k == 4 && c != '-') {
                    for (int j = 0; j < k - 1; j++) str1[j] = str1[j +
1];
                    str1[k - 1] = ' ';
                    k--;
                    str1 = (char*) realloc(str1, k * sizeof(char));
                    flag = 0;
                }
            }
            else if (k == 4 && c == '-' && r > 4 && arr[r - 5] == '\n') {
                for (int j = 0; j < r - 4; j++) {

```

```

        str[w] = arr[j];
        w++;
    }
    explicit_bzero(arr, 0);
    r = 4;
    arr = (char*) realloc(arr, r * sizeof(char));
    memcpy(arr, str1, 4);
    flag = 1;
    }
    }
    else {
        if (y == 4 && k == 17 && (c == ' ' || c == '\n') && flag) {
//проверка, что строка str1 вида xxx-x-xxx-xxxxx-x
        char *str2 = NULL; //вспомогательная строка (её будем
разбивать по "-")

        str2 = (char*) realloc(str2, k * sizeof(char));
        memcpy(str2, str1, k);
        char *p = strtok(str2, "-");
        int f = 1;
        while (p) {
            if (g == 0 && strlen(p) != 3) f = 0;
            else if (g == 1 && strlen(p) != 1) f = 0;
            else if (g == 2 && strlen(p) != 3) f = 0;
            else if (g == 3 && strlen(p) != 5) f = 0;
            g++;
            if (f == 0) break;
            else p = strtok(NULL, "-");
        }
        explicit_bzero(str2, 0);
        free(str2);
        if (f == 1) { //проверка по алгоритму ISBN-13
            int d = 0, l = 0;
            for (int j = 0; j < 15; j++) {
                if (j != 3 && j != 5 && j != 9) {
                    l++; //для того, чтобы умножать на
3 только четные элементы

                    d += 2 * ((l + 1) % 2) * ((int) str1[j] -
48) + (int) str1[j] - 48; //48 - код 0 в ASCII
                }
            }
            if (d % 10 != ((int) str1[16] - 48)) f = 0;
            else { //нужно увеличить строку str на 2 для того,
чтобы можно было обособить подходящую нам str1, и вставить ее в исходную
                i += 2;
                str = (char*) realloc(str, i * sizeof(char));
                str[i - 2 - r - 1] = '*';
            }
        }
    }
}

```

```

        memcpy(str + i - 2 - r, arr, r);
        str[i - 2] = '*';
        str[i - 1] = c;
        w += 3 + r;
        //очистим str1
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
        //очистим arr
        explicit_bzero(arr, 0);
        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
}
if (f == 0) {
    //записываем в str то, что лежит в str1 (так как так
лежит то, что нам не подошло)

        memcpy(str + i - r - 1, arr, r);
        str[i - 1] = c;
        w += r + 1;
        //очистим str1
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
        //очистим arr
        explicit_bzero(arr, 0);
        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
}
else { //записываем в str то, что лежит в str1 (так как так лежит
то, что нам не подошло)

        memcpy(str + i - r - 1, arr, r);
        str[i - 1] = c;
        w += r + 1;
        //очистим str1
        explicit_bzero(str1, 0);
        str1 = (char*) realloc(str1, sizeof(char));
        k = 0;
        //очистим arr
        explicit_bzero(arr, 0);
        arr = (char*) realloc(arr, sizeof(char));
        r = 0;
    }
    y = 0;
}
}
else {

```

```

        if (r != 0) {
            memcpy(str + i - r - 1, arr, r);
            w += r;
            explicit_bzero(str1, 0);
            str1 = (char*) realloc(str1, sizeof(char));
            k = 0;
            explicit_bzero(arr, 0);
            arr = (char*) realloc(arr, sizeof(char));
            r = 0;
        }
        str[i - 1] = c;
        w++;
        if (c == '\n') countn++;
    }
}

if (r != 0) memcpy(str + w, arr, r);
//запись строки в файл
if (strlen(txt2) != 0) {
    FILE *f1 = fopen(txt2, "w");
    //покраска
    if (cv != 0) {
        char *pp = strtok(str, "*");
        int a;
        if (str[0] == '*') a = 0;
        else a = 1;
        while (pp) {
            a++;
            if (a % 2 != 0) fprintf(f1, "\e[35m%s\e[m", pp);
            else fprintf(f1, "%s", pp);
            pp = strtok(NULL, "*");
        }
    }
    else fputs(str, f1);
    fclose(f1);
}

else {
    //покраска
    if (cv != 0) {
        char *pp = strtok(str, "*");
        int a;
        if (str[0] == '*') a = 0;
        else a = 1;
        while (pp) {
            a++;
            if (a % 2 != 0) printf("\e[35m%s\e[m", pp);
            else printf("%s", pp);
            pp = strtok(NULL, "*");
        }
    }
}

```

```

        }
    }
    else printf("%s", str);
}
explicit_bzero(str, 0);
explicit_bzero(str1, 0);
explicit_bzero(arr, 0);
free(str);
free(str1);
free(arr);
fclose(f);
}

```

infileonly.c:

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

```

//обрабатывает только целые последовательности (в файле)

```

void infileonly(char *txt1, char *txt2, int cv, int bv, int ev) {
    FILE *f = fopen(txt1, "r");
    //str - вводимая строка, str1 - вспомогательная строка, которая будет хранить в себе
    //нужную нам подстроку (или что-то похожее на неё)
    char *str = NULL;
    char *str1 = NULL;
    str = (char*) malloc(sizeof(char));
    str1 = (char*) malloc(sizeof(char));
    //для считывания символов
    int t;
    char c;
    int i = 0, k = 0; //i - текущая длина str, k - текущая длина str1
    int y = 0, g = 0; //счетчики
    int countn = 1; //счетчик переносов строки, чтобы следить за номером строки
    while ((t = fgetc(f)) != EOF) {
        //обработка переноса строки и других символов
        if (t == 13) continue;
        else if (t == 10) c = '\n';
        else c = (char) t;
        i++;
        str = (char*) realloc(str, i * sizeof(char)); //увеличим строку
        if ((bv == 1 || countn >= bv) && (ev == 0 || ev >= countn)) { //обработка
        конкретных строк (ограничения b и e)
            if (isdigit(c) || c == '-') { //будем здесь собирать строки с числами
                k++;
                str1 = (char*) realloc(str1, k * sizeof(char)); //увеличим строку
                str1[k - 1] = c;
                if (c == '-') y++; //счетчик символов '-'
            }
        }
    }
}

```



```

else {
    if (c == '\n') countn++;
    if (y == 4 && k == 17 && (c == ' ' || c == '\n')) { //проверка, что
строка str1 вида xxx-x-xxx-xxxxx-x
        char *str2 = NULL; //вспомогательная строка (её будем
разбивать по "-")

        str2 = (char*) realloc(str2, k * sizeof(char));
        memcpy(str2, str1, k);
        char *p = strtok(str2, "-");
        int f = 1;
        while (p) {
            if (g == 0 && strlen(p) != 3) f = 0;
            else if (g == 1 && strlen(p) != 1) f = 0;
            else if (g == 2 && strlen(p) != 3) f = 0;
            else if (g == 3 && strlen(p) != 5) f = 0;
            g++;
            if (f == 0) break;
            else p = strtok(NULL, "-");
        }
        free(str2);
        if (f == 1) { //проверка по алгоритму ISBN-13
            int d = 0, l = 0;
            for (int j = 0; j < 15; j++) {
                if (j != 3 && j != 5 && j != 9) {
                    l++; //для того, чтобы умножать на
3 только четные элементы
                    d += 2 * ((l + 1) % 2) * ((int) str1[j] -
48) + (int) str1[j] - 48; //48 - код 0 в ASCII
                }
            }
            if (d % 10 != ((int) str1[16] - 48)) f = 0;
            else { //нужно увеличить строку str на 2 для того,
чтобы можно было обособить подходящую нам str1, и вставить ее в исходную
                i += 2;
                str = (char*) realloc(str, i * sizeof(char));
                str[i - 2 - k - 1] = '*';
                memcpy(str + i - 2 - k, str1, k);
                str[i - 2] = '*';
                str[i - 1] = c;
                //очистим str1
                explicit_bzero(str1, 0);
                str1 = (char*) realloc(str1, sizeof(char));
                k = 0;
            }
        }
    }
    if (f == 0) {

```

```

//записываем в str то, что лежит в str1 (так как так
лежит то, что нам не подошло)

memcpy(str + i - k - 1, str1, k);
str[i - 1] = c;
//очистим str1
explicit_bzero(str1, 0);
str1 = (char*) realloc(str1, sizeof(char));
k = 0;
    }
}
else { //записываем в str то, что лежит в str1 (так как так лежит
то, что нам не подошло)

memcpy(str + i - k - 1, str1, k);
str[i - 1] = c;
//очистим str1
explicit_bzero(str1, 0);
str1 = (char*) realloc(str1, sizeof(char));
k = 0;
    }
    y = 0;
}
}
else {
    str[i - 1] = c;
    if (c == '\n') countn++;
}
}
//запись строки в файл
if (strlen(txt2) != 0) {
    FILE *f1 = fopen(txt2, "w");
    //покраска
    if (cv != 0) {
        char *pp = strtok(str, "*");
        int a;
        if (str[0] == '*') a = 0;
        else a = 1;
        while (pp) {
            a++;
            if (a % 2 != 0) fprintf(f1, "\e[35m%s\e[m", pp);
            else fprintf(f1, "%s", pp);
            pp = strtok(NULL, "*");
        }
    }
    else fputs(str, f1);
    fclose(f1);
}
else {
    //покраска

```

```

    if (cv != 0) {
        char *pp = strtok(str, "*");
        int a;
        if (str[0] == '*') a = 0;
        else a = 1;
        while (pp) {
            a++;
            if (a % 2 != 0) printf("\e[35m%s\e[m", pp);
            else printf("%s", pp);
            pp = strtok(NULL, "*");
        }
        else printf("%s", str);
    }
    explicit_bzero(str, 0);
    explicit_bzero(str1, 0);
    free(str);
    free(str1);
    fclose(f);
}

```